A Unified Malware Classification Approach For Heterogeneous Consumer Devices

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—Detecting malware across several domains and operating systems faces significant challenges due to the differences in system structure as well as high computation costs. While some recent methods have achieved successful malware detection through the use of computer vision techniques by mapping raw binary files into two-dimensional images, the transformation generates artificial spatial locality and breaks the sequential nature of the bytecode. In this project, we propose a cross-domain malware classification framework using a Hierarchical Mixture of Experts (MoE) architecture with 1-Dimensional Convolutional Neural Networks (1D-CNNs). To maximize computational efficiency and signal density, raw executables are segmented and only the first and last 2^{16} bytes are used while discarding the structurally noisy middle sections.**

The 1D sequence is passed through a Convolutional Neural Network (CNN) and subsequently a gating network routes the extracted features to a domain specific expert (Android, Linux or Windows), effectively eliminating negative transfer. Evaluating 11,583 Android, 20,000 Linux, and 22,207 Windows samples, our framework achieved a weighted average F1 score of 0.9646. Moreover, to ensure model explainability, we used GradCAM [5] to visualize network activations, effectively localizing malicious signatures. Our findings demonstrate that segmented 1D sequence of high signal combined with MoE routing model achieves a highly accurate and computationally lean solution for cross-domain malware classification.

***Index Terms*—Security, Cross-domain, Malware detection, Malware classification, Deep learning, Hierarchical experts.**

I. Introduction

MALWARE or malicious software has been on an exponential proliferation in recent years. As of recent threat analysis, the volume of malware on desktop and mobile ecosystem continue to escalate and about 450,000+ new malware samples are detected everyday [1]. The malwares also get increasingly sophisticated, and this remains a critical challenge in modern cybersecurity.

Traditional signature based detection mechanisms are falling behind due to techniques like advanced obfuscation, polymorphism, string encryption, and packing. Good feature engineering remains an open problem in the literature due to the challenging task of characterizing maliciousness in terms of app artefacts [2]. Consequently, the community have moved to deep learning approaches, notably image based convolution techniques mainly because of the recent development in Computer Vision. Nataraj et. al. demonstrated that converting raw executable binaries into grayscale images allows Convolutional Neural Networks (CNNs) to classify structural malicious patterns [3].

STAMINA [4] from Microsoft and Intel labs have shown that image based architectures achieve considerable success, but they rely on reshaping sequential byte stream into two dimensional images. This inherently causes distortion and adds artificial locality where neighboring columns lack any true relationship in the original executable.

Furthermore, classifying malware of different domains with one single neural network leads to negative transfer, wherein the learned structural features of one domain degrades the model performance on another domain.

To address these limitations, this project proposes a cross-platform malware classification framework utilizing a Hierarchical Mixture of Experts (MoE) using by One-Dimensional Convolutional Neural Networks (1D CNNs). By processing raw bytes as 1D sequence, the framework preserves the spatial locality of the executables, and to eliminate negative transfer, the framework uses distinct experts for each domain with a separate gating network that routes the extracted features to the correct expert.

Furthermore, this research tackles the inherent drawbacks in static deep learning malware analyzers, particularly excessive computation due to large multi-megabytes in size executables, as well as the lack of transparency due to their blackbox nature. Firstly, the data preprocessing stream utilizes segmentation of raw binaries by only using the first and last 2^{16} bytes of each file and ignoring the noisy middle parts, which dramatically decreases the computational needs while preserving all necessary signals. Secondly, the method includes the application of 1D Grad-CAM [5], providing the visual interpretation that the MoE routing mechanism and 1D CNN experts successfully focus on malicious indicators located in the file headers and footers.

This research aims to address the following key questions:

- **RQ1:** Can processing raw executables as continuous 1D byte sequences, rather than 2D images, better preserve structural locality and improve malware classification accuracy?
- **RQ2:** How can a deep learning architecture be designed to classify malware across heterogeneous operating systems (Android, Linux, Windows) without suffering from negative transfer?
- **RQ3:** Does intelligent byte segmentation reduce computational overhead while maintaining high classification fidelity?
- **RQ4:** How can the decision-making process of the proposed deep learning model be interpreted and visualized to identify critical malicious signatures?

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

0000–0000/00$00.00 © 2021 IEEE

TABLE I
COMPARISON OF STATE-OF-THE-ART MALWARE CLASSIFICATION FRAMEWORKS AGAINST THE PROPOSED HIERARCHICAL MIXTURE OF EXPERTS (MOE) ARCHITECTURE. THE TABLE HIGHLIGHTS KEY ARCHITECTURAL DIFFERENCES, INCLUDING IMAGE REPRESENTATION, PLATFORM SCOPE, DIMENSIONALITY (1D VS. 2D), FEATURE EXTRACTION STRATEGIES (FULL BINARY VS. SEGMENTATION), AND THE PRESENCE OF DYNAMIC ROUTING MECHANISMS.

Framework	STAMINA [4]	DEXRAY [2]	Kumar et al. [6]	MalSFF [7]	Proposed Framework
Image Repr.	2D Matrix	1D Vector	1D Vector	Fused Features	1D Vector
Platform Scope	Windows PE	Android DEX	Platform Specific	Multi-OS	Android, Linux & Windows
Dimensionality	2D CNN	1D CNN	1D CNN	Hybrid	1D CNN
Architecture	Monolithic	Monolithic	Stacked Ensemble	Feature Fusion	Hierarchical MoE
Extracted Seg.	Full Binary	Full Binary	Full Binary	Extracted Features	2×2^{16} bytes (Head+Tail)
Routing	No	No	No	No	Yes (Dynamic)

The main contributions of this work are summarized as follows:

- **A Hierarchical Mixture of Experts (MoE) Architecture:** We propose a cross-platform malware classification framework utilizing a 1D CNN-based MoE architecture with a dynamic gating network. This routes extracted features to domain-specific experts (Android, Linux, Windows), effectively eliminating negative transfer.
- **1D Sequence Preservation and Byte Segmentation:** By processing raw bytes as 1D sequences, the framework preserves the spatial locality of executables. To tackle computational overhead, we introduce a segmentation strategy that discards noisy middle sections, utilizing only the first and last 2^{16} bytes of each file.
- **Model Explainability via 1D Grad-CAM:** We integrate 1D Gradient-weighted Class Activation Mapping (Grad-CAM) to provide visual interpretation, proving that the MoE routing mechanism and 1D CNN experts successfully focus on malicious indicators located in the file headers and footers.
- **Comprehensive Evaluation:** We evaluate the framework on a highly heterogeneous dataset of 53,790 samples across 31 classes, achieving a weighted average F1-score of 0.9646, and conduct an ablation study to validate the segmentation thresholds and model robustness against adversarial masking.

A. Objective

The primary objective of this project is to engineer *a unified, cross-domain malware classification framework capable of accurately identifying malicious executables across heterogeneous domains / operating systems without suffering from negative transfer.*

To achieve this goal, the project targets the following technical sub objectives:

- **Structural Preservation**: To avoid 2D image reshaping paradigm and process raw binaries as 1D sequence vectors, thus preserving the native structural locality of the binaries.
- **Architectural Isolation**: To implement a Hierarchical Mixture of Experts (MoE) containing a dynamic gating network that dynamically routes data to specialized 1D-CNN experts.
- **Computational Optimization**: To reduce training overhead and inference latency by discarding noise, and extracting only the most high signal segments of the executable.
- **Model Explainability**: To integrate 1D Gradient-weighted Class Activation Mapping (Grad-CAM) to visualize features of the image that the model uses in its decision making.

II. RELATED WORKS

In the field of malware classification using deep learning models trained on raw binary data, there have been significant advancements, progressing from traditional static analyses to computer vision and sequence modeling approaches. The foundational approach to treating malware classification as a visual recognition task was popularized by Nataraj et al. [3], who demonstrated that reshaping raw binaries into 2D grayscale images revealed distinct structural textures indicative of specific malware families. This methodology was heavily scaled by Chen et al. in the STAMINA framework , utilizing 2D reshaping matrices, standard 2D CNN feature extractors, and static byte representation. However, none of these 2D methods provide a way to ensure the preservation of the integrity of 1D strings. By representing a constant flow of bytes as a fixed-size square grid, such systems artificially create spatial proximity between elements that, in reality, are unrelated in a one-dimensional flow. Additionally, they process an entire binary without any form of segmentation, resulting in considerable computational inefficiency.

Recognizing the structural distortion caused by 2D reshaping, recent literature has pivoted toward 1D representations. Daoudi et al. proposed DEXRAY [2], which successfully modeled Android bytecode as a continuous 1D vector, empirically proving that preserving sequence integrity outperforms 2D spatial methods. Similarly, Kumar et al. (2024) [6] demonstrated the computational efficiency of combining 1D-CNNs with stacked ensemble techniques. While these architectures utilize 1D CNNs and vector-based representation for high-fidelity local feature extraction, they show poor generalizability across platforms. For instance, DEXRAY is specific to

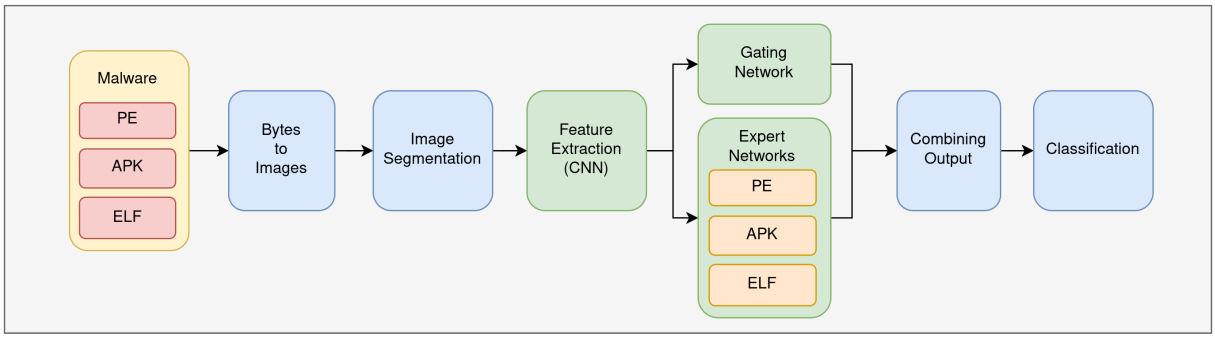


Fig. 1. Pipeline of the proposed cross-platform malware classification framework. Raw binaries from different domains are first converted into continuous 1D images. To optimize computational efficiency while preserving structural integrity, the streams undergo segmentation to extract the high-signal first and last 2^{16} bytes. A 1d-CNN is used for feature extraction and a dynamic gating network evaluates the feature map to determine the structural domain and routes the data to specialized expert network. The final classification is derived by combining the routing probabilities with the expert outputs, effectively eliminating cross-domain negative transfer.

the Android (DEX) operating system. Moreover, they cannot perform dynamic routing, which means that when used for a mixed dataset (Android, Linux, and Windows), the network must learn contradictory structures simultaneously.

To address platform heterogeneity, Kumar and Kumar (2026) introduced MalSFF [7], utilizing multi-static feature fusion for multi-architecture detection. Foundational machine learning theory, such as Jacobs et al.'s Adaptive Mixtures of Local Experts [8] and Shazeer's sparsely-gated MoE [9], provides a mathematical basis for routing conflicting data to specialized subnetworks. While MalSFF successfully handles multi-OS datasets through feature fusion and ensemble predictions, it relies on image visualization and the fusion of static features rather than dynamically routing inputs to isolated, domain-specific experts. The architecture forces heterogeneous data through a shared fusion process, leaving the model susceptible to negative transfer when attempting to learn contradictory structural patterns simultaneously. Furthermore, by utilizing fused extracted features instead of targeted raw 1D sequence segmentation, the method fails to preserve the continuous native spatial locality of the binaries and does not mitigate the computational overhead of processing unsegmented executables.

The proposed framework combines the distinct advantages of the state-of-the-art without any of the inherent structural and computational inefficiencies. Table I provides a comprehensive comparison of the existing architectures and the newly designed architecture.

III. Methodology

The pipeline starts from binary executables and ends with the predicted malware class. The complete pipeline is shown in Fig. 1. This section explains in detail each step of the pipeline from start to finish.

A. Dataset

To perform an analysis of the proposed cross-platform Hierarchical Mixture of Experts (MoE) architecture, a highly heterogeneous dataset is needed. The complete dataset comprises 53,790 files, which includes Android, Linux, and Windows binaries.

1) Android Dataset: The Android part of the dataset consists of 11,583 APKs, gathered from well-known cybersecurity data repository CICMalDroid2020 [10]. The samples are split into five different behavior groups, namely Adware (1,253), Banking Trojans (2,087), Benign (1,795), Riskware (2,546), and SMS Malware (3,902). (see Table II)

2) Linux Dataset: The Linux malware dataset consists of 20,000 Executable and Linkable Format (ELF) collected from MalwareBazaar [13]. The ELFs are classified based on the signatures provided by MalwareBazaar itself. Classes include CoinMiner, DDoSAgent, Gafgyt, Hajime, Kaiji, Mirai, MooBot, Prometei, SSHdKit, and Tsunami. Table III shows the number of samples in each class.

3) Windows Dataset: Windows dataset includes 22,207 Portable Executable (PE) samples collected from Malhub [11] repository. The dataset demonstrates a high level of diversity, consisting of 16 unique malware families. Table IV shows the full list of classes and the number of samples in each.

4) Dataset Imbalance: In accordance with the real-world practice of cybersecurity researches, the whole dataset shows inherent class imbalance both within each OS and between the OSs. Unlike standard solutions, such as oversampling of datasets (introducing artificial structural noise into data), the problem of imbalance is solved by using the MoE architecture. The dynamic gating network of MoE separately weighs probabilities for each platform, therefore avoiding any potential biases.

B. Data preprocessing and 1D vectorization

Converting raw binary data to 1D grayscale image is a straightforward process. 1D grayscale images are stored as array of integer values ranging from 0 to 255, this maps perfectly to the amount of information contained in a single byte. Consequently, each byte of the binary is represented as a pixel value as shown in Fig. 2.

Our dataset consists of malware samples in the form of Android Application Packages (APKs), Linux Executable and

TABLE II
DISTRIBUTION OF THE ANDROID MALWARE AND BENIGN DATASET SOURCED FROM THE CICMALDROID2020 REPOSITORY. THE DATASET COMPRISES 11,583 APK SAMPLES CATEGORIZED INTO FIVE DISTINCT BEHAVIORAL CLASSES. THIS CLASS IMBALANCE REFLECTS REAL-WORLD DISTRIBUTIONS AND PROVIDES A COMPREHENSIVE REPRESENTATION OF THE MODERN MOBILE THREAT LANDSCAPE FOR EVALUATING THE DOMAIN-SPECIFIC EXPERT.

Class Name	Sample Count
Sms_final	3,902
Riskware_final	2,546
Banking_final	2,087
Benign_final	1,795
Adware_final	1,253

TABLE III
DISTRIBUTION OF THE LINUX EXECUTABLE AND LINKABLE FORMAT (ELF) DATASET COMPRISING 20,000 SAMPLES. THIS DATASET ENCOMPASSES 10 DIFFERENT PROMINENT LINUX-BASED MALWARE SIGNATURES.

Class Name	Count	Class Name	Count
Mirai	14,357	Kaiji	578
Gafgyt	1,795	CoinMiner	476
Prometei	900	Tsunami	311
Hajime	654	MooBot	177
SSHdKit	611	DDoSAgent	141

TABLE IV
DISTRIBUTION OF THE WINDOWS PORTABLE EXECUTABLE (PE) DATASET COLLECTED FROM THE MALHUB REPOSITORY. THE COLLECTION INCLUDES 22,207 SAMPLES SPANNING 16 UNIQUE MALWARE FAMILIES. THIS DATASET CAPTURES THE HIGH STRUCTURAL DIVERSITY AND HETEROGENEITY OF DESKTOP-BASED MALWARES.

Class Name	Count	Class Name	Count
winwebsec	3,651	delfinject	1,146
vundo	1,793	startpage	1,135
zbot	1,785	zeroaccess	1,119
hotbar	1,491	fakerean	1,063
obfuscator	1,445	cycbot	1,029
alureon	1,328	rbot	1,017
onlinegames	1,293	ceeinject	894
bho	1,176	agent	842

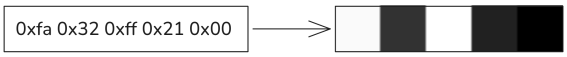


Fig. 2. Visualization of the raw binary-to-image conversion process. Each sequential byte of the executable, represented by a hexadecimal value ranging from 0x00 to 0xFF (0 to 255 in decimal), is directly mapped to a corresponding 8-bit grayscale pixel intensity.

Linkable Formats (ELFs), and Windows Portable Executables (PEs). ELFs and PEs are native binary executable themselves, thus a simple algorithm as shown in Algorithm 1 is used. It reads a file as byte stream and generates an 8-bit grayscale vector and saves the vector as a PNG. APKs are not binary executables themselves but compressed ZIP archives, and therefore they must be extracted before converting. The executables in APKs are stored as Dalvik Executable in the `classes.dex` file within the APK. Larger APKs may contain more than one `classes.dex` (they are conveniently named as `classes1.dex`, `classes2.dex` and so on), in which case all the Dalvik Executables are concatenated sequentially before converting to images.

To test the efficiency of our 1D approach, a traditional 2D image baseline was established for direct comparison. Drawing inspiration from established papers like DEXRAY [2], the bytestream was converted to a square image and resized to 128x128 pixels. Streams that lacked a perfect-square length were padded with zeros (0x00), as detailed in Algorithm 2. Specifically, lines 3 to 5 read a file as a byte stream, determine its length, and calculate the ceiling of the square root of that length. Line 6 squares this value to find the nearest perfect square that is larger than or equal to the actual file length. Lines 7 to 9 pad the byte stream with zeros until its length matches this perfect square. Finally, line 10 converts the byte stream into a grayscale square image, lines 11 and 12 resize the output to 128x128 pixels and save it in PNG format.

Algorithm 1 Binary file to 1D grayscale image

1: **Input**: Binary file (n bytes)
2: **Output**: 8-bit grayscale vector image of size $(1, n)$
3: bytestream $\leftarrow$ file.toByteStream()
4: ln $\leftarrow$ bytestream.length()
5: img $\leftarrow$ generate8bitGreyScaleVectorImage(bytestream, ln)
6: img.save()

Algorithm 2 Binary file to 2D grayscale image

1: **Input**: Binary file (n bytes)
2: **Output**: 8-bit grayscale vector image of size $(128, 128)$
3: bytestream $\leftarrow$ file.toByteStream()
4: ln $\leftarrow$ bytestream.length()
5: sqrt $\leftarrow \lceil \sqrt{\text{ln}} \rfloor$
6: sq $\leftarrow$ sqrt $\times$ sqrt
7: **while** bytestream.length() $\neq$ sq **do**
8: bytestream $\leftarrow$ bytestream + "\x00"
9: **end while**
10: img $\leftarrow$ generate8bitGreyScaleSquareImage(bytestream, sqrt)
11: img $\leftarrow$ img.resize(height $= 128$, width $= 128$)
12: img.save()

C. Image Segmentation and Feature Extraction

Size of malware samples ranges from a few kilobytes to tens of megabytes, and in-order to run them through a convolution model, it must be converted to a fixed length. Fixed size resizing is what most researchers resort to [2], [4], but comes with the caveat that features in larger samples get compressed while features in smaller samples get enlarged. We, instead, chose to segment the image and only run the model on high signal segments and disregard the noisy sections.

Instead of standard 2D resizing, which artificially distorts spatial locality, we segment the raw binaries to isolate high-signal regions and discard structural noise. Specifically, we extract and concatenate the first and last 2^{16} bytes (creating a 1D vector of 131,072 bytes) as seen in Fig. 3. This focuses the model on critical headers and footers where malicious payloads and imports are most concentrated. The empirical justification for selecting the 2^{16} byte threshold and these specific positional segments is detailed later in our Ablation Study.

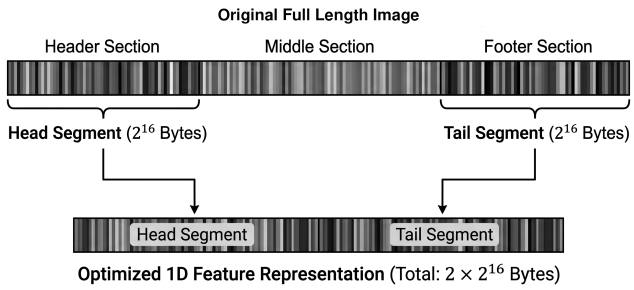


Fig. 3. Segmenting 1D images

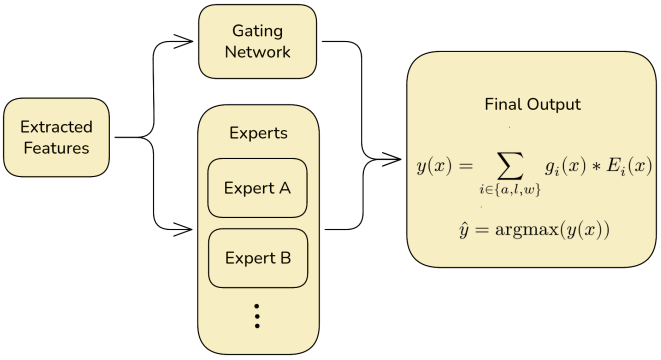


Fig. 4. Architecture of the Hierarchical Mixture of Experts (MoE) routing mechanism. The 1D-CNN extracted feature vectors are fed simultaneously into a dynamic gating network and a set of domain-specific experts. The gating network computes a confidence weight distribution $g_i(x)$ across the three domains (Android, Linux, and Windows). These weights are then multiplied by the individual probability outputs $E_i(x)$ of their respective experts. The final classification prediction $\hat{y}$ is determined by applying an argmax function to the weighted sum of all expert outputs.

This segmented vector is then inputted to a custom 1-Dimensional Convolutional Neural Network (1D-CNN) to perform extraction of malicious features in the structure. The first block of extraction layers utilizes a large receptive field to extract broad structural information in the byte stream. This includes passing the stream through a Conv1d layer with 16 filters, kernel size 7, and padding 3, followed by Batch normalization and ReLU activation, and substantial down-sampling using a MaxPool1d layer with kernel size 4 and stride 4. This same architectural design repeats itself in the second block, but this time with 32 convolution filters.

The convolutional filters keep shrinking as the data progresses through each of the network layers. The third block applies a Conv1d layer of 64 filters (with kernel size 5, padding 2) and further MaxPool1d down-sampling with kernel size 4 and stride 4. This is followed by yet another Conv1d layer of 128 filters (kernel size 5, padding 2). A dropout layer with rate of 0.3 is used to prevent overfitting of the data, after which flattening occurs to produce a dense representation of high dimensionality. This extracted feature map is routed to the gating network.

D. Hierarchical MoE architecture

Processing all Android DEXs, Linux ELFs, and Windows PEs through a singular, monolithic classifier inevitably induces negative transfer, the patterns of the learned bytes of one operating system to confuse the classification of the other. To separate platform-specific feature extraction, we use a Hierarchical Mixture of Experts (MoE) network with a dynamic gating scheme and three platform-specific expert networks.

1) Dynamic Gating Network: The gating network acts as a routing mechanism, utilizing a separate Deep Neural Network (DNN) to determine the domain of the binary. It accepts the flattened feature vector ($x \in \mathbb{R}^d$) from the feature extraction module and processes it through a 2-Layer Multi-Layer Perceptron (MLP) featuring batch normalization and dropout.

Let W_{g1} and W_{g2} be the learned weight matrices, and b_{g1} and b_{g2} the biases of the first and second linear layers, respectively. The intermediate hidden representation h_g is calculated as:

$$h_g = \text{Dropout}(\text{ReLU}(\text{BatchNorm}(W_{g1} \cdot x + b_{g1}))) \quad (1)$$

The final gating output $G(x)$ is then computed by applying a Softmax function over the second linear transformation:

$$G(x) = \text{Softmax}(W_{g2} \cdot h_g + b_{g2}) \quad (2)$$

This generates a confidence weight distribution $G(x) = [g_a(x), g_l(x), g_w(x)]$, where g_a, g_l, and g_w represent the probability that the input binary belongs to Android, Linux, or Windows respectively.

2) Domain Specific Experts: Each domain (Android, Linux, & Windows) has a separate MLP model that learns the features of the malwares from their respective classes. Domain-specific models can differ in architecture to accommodate the complexity of malwares in the domain, though in this framework we utilize three structurally similar MLPs with two hidden layers for each operating system.

The expert models take the same extracted features as the gating network and output the probability of each sub-class.

$$h_1 = \text{ReLU}(W_{i1} \cdot x + b_{i1}) \quad (3)$$

$$h_2 = \text{Dropout}(\text{ReLU}(W_{i2} \cdot h_1 + b_{i2})) \quad (4)$$

$$E_i(x) = \text{Softmax}(W_{i3} \cdot h_2 + b_{i3}) \quad (5)$$

Where W_{i1}, W_{i2}, & W_{i3} are the weights, and b_{i1}, b_{i2}, & b_{i3} are the biases of the ith expert. $E_i(x)$ yields the probabilities of the sub-classes (e.g., the specific malware family or benign status) according to that specific expert's learned parameters.

3) Expert Weighting and Final Classification: The architecture then calculates the final classification as the product of the routing probabilities and the expert outputs. The model is based on a soft-weighting mechanism, as opposed to a hard routing decision (which makes backpropagation much easier).

The probability that the malware belongs to a specific class is the product of the weight of the expert and the predicated probability of that class.

$$y(x) = \sum_{i \in \{a,l,w\}} g_i(x) \cdot E_i(x) \quad (6)$$

This mathematical formulation ensures that the expert with the highest structural confidence proportionally dominates the final decision matrix. The final class prediction y' is derived by selecting the class with the highest overall score.

$$\hat{y} = \text{argmax}(y(x)) \tag{7}$$

4) Objective Function and Loss Formulation: Multi-class classification tasks traditionally rely on the standard Cross-Entropy loss function. In PyTorch, the `nn.CrossEntropyLoss` module is designed to accept raw logits, applying an internal `LogSoftmax` function. However, because the final output of the proposed architecture consists of normalized probability values rather than raw logits, utilizing standard Cross-Entropy would result in a redundant softmax operation that degrades gradient calculations.

Thus, Negative Log-Likelihood Loss (`nn.NLLLoss`) is employed as the objective function. Since `nn.NLLLoss` explicitly requires log-probabilities, a logarithmic transformation is applied to the model's final probability outputs prior to calculating the loss.

E. Model Optimization and Explainability

For maximum performance efficiency and mathematical transparency of classifications made by the proposed model, it has been optimized with respect to hyperparameters and features a gradient-based visualization of its predictions.

1) Hyperparameter Optimization and Baseline Classification: As it was mentioned above, the performance of deep learning pipelines is highly dependent on the parameter settings. Specifically, since our pipeline includes complex Mixture of Experts architecture, automatic identification of the ideal configuration would eliminate the risk of human error during manual configuration and would help us choose the best set of parameters. Therefore, we applied a hyperparameter optimizer to find the best possible setting in terms of minimal classification loss, including optimal learning rates, dropout probability values, and batch sizes.

Also, in order to confirm the positive impact of the preservation of input data structure, we ran our optimally tuned 1D-CNN experts model and the equivalent but baseline 2D-CNN model. Equivalence means identical amount of layers, number of filters, and hyperparameter tuning settings. In case if the resulting difference in F1-scores occurred, it could be safely attributed to the dimensionality of the input vector.

2) Application of 1D Grad-CAM and Data Segmentation Confirmation: One of the main critiques towards machine learning models is the black-box nature of the decision-making process. That is why it is vital to incorporate explanation features into the model's architecture in order to make decisions transparent. We employed a 1-dimensional version of Gradient-weighted Class Activation Mapping (1D Grad-CAM) [5] to achieve this goal.

Grad-CAM technique involves computation of gradients of the target concept flowing into the last convolutional layer of the network in order to form a coarse localization map of parts of input vector being critical for the final prediction. Projection of this map onto the byte vector will help to visualize and identify exactly which byte sequences led to the classification of the sample as malware.

F. Adversarial Masking Resilience

To evaluate the robustness of the proposed Hierarchical Mixture of Experts (MoE) and its specialized 1D CNN feature extractors against adversarial perturbations, a targeted adversarial masking experiment is integrated into the evaluation. Deep learning models in static malware analysis are frequently susceptible to obfuscation techniques that deliberately hide or modify critical malicious signatures.

To simulate such an evasion attack, the 1D Grad-CAM implementation is utilized to isolate the exact spatial locations of the features that contribute most heavily to the model's positive predictions. An adversarial masking technique is then applied by systematically zeroing out the top $x\%$ of these highly predictive bytes. The model's classification performance is subsequently re-evaluated across multiple masking thresholds, specifically for $x \in \{5\%, 10\%, 15\%\}$. This experiment empirically quantifies the architecture's structural resilience and determines how much targeted obfuscation the model can withstand before classification fidelity degrades. The quantitative findings of this masking analysis are detailed in the Results section.

G. Evaluation Protocol

For the evaluation of the architecture, the whole dataset consisting of 53,790 malware binaries was split into an 80% training set, 10% validation, and 10% isolated testing set. The use of accuracy as a measure is inappropriate since the class distribution in the merged dataset is highly imbalanced; for example, the Sms_final class significantly outnumbers minorities like agent.

Therefore, the weighted average F1-score was selected as the principal evaluation criterion. By calculating the harmonic mean of precision and recall, the F1-score simultaneously penalizes both false positives and false negatives, providing a robust performance metric for imbalanced data. The weighted variant further accounts for dataset disparity by scaling each class's F1-score by its proportion of actual instances.

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \tag{8}$$

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \tag{9}$$

Where TP = True Positives, FP = False Positives, and FN = False Negatives.

$$\text{F1}_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{10}$$

$$\text{Weighted F1} = \sum_{c=1}^{C} \left(\frac{N_c}{N} \times \text{F1}_c \right) \tag{11}$$

Where C is the total number of classes, N_c is the number of actual samples in class c, and N is the total number of samples across the entire dataset.

TABLE V
OPTIMAL HYPERPARAMETER CONFIGURATION DETERMINED VIA OPTUNA. THE TABLE DETAILS THE DEFINED SEARCH SPACE AND THE FINAL SELECTED VALUES FOR THE OPTIMIZER, LEARNING RATE, BATCH SIZE, AND WEIGHT DECAY THAT EFFECTIVELY MINIMIZED VALIDATION LOSS FOR THE 1D-CNN EXPERTS AND GATING NETWORK.

Hyperparameter	Search Space	Final Optimized Value
Optimizer	Adam, AdamW	Adam
Learning Rate	10^{-5} to 10^{-2}	5×10^{-3}
Batch Size	32, 64, 128, 256	128
Weight Decay	10^{-6} to 10^{-3}	10^{-5}

TABLE VI
FIXED HYPERPARAMETER CONFIGURATIONS APPLIED DURING THE MODEL TRAINING PHASE.

Hyperparameter	Value
No. of Epochs	100
Loss	Negative Log Likelihood Loss
Train, Val, Test Split	80:10:10

IV. RESULTS AND EVALUATION

A. Implementation Details and Experimental Setup

In order to facilitate the perfect reproducibility of the presented 1D MoE architecture, this section presents an exact description of the utilized software stack and the configuration settings which proved to be optimal in the process of model training and evaluation.

The data ingestion, bytestream to image conversion, and 1D byte-segmentation were implemented using a custom rust-script, chosen for its speed and memory-safety. The deep learning architecture was made entirely using PyTorch. We also used Optuna [12] to find the optimal hyperparameters. Table V details the final, optimal configurations used to generate the evaluation metrics.

B. Gating Network Performance

Before evaluating the final malware classification, it is critical to assess the dynamic gating network used for routing the binaries. The gating network successfully separated the heterogeneous data, achieving a flawless routing accuracy of 1.0000 across 5,379 testing samples.

The classification report highlights a perfect weighted average F1-score of 1.0000, with absolute precision (1.0000) across all three domains: Android, Linux, and Windows (see Table VII and Fig. 5). The confusion matrix provided below confirms this strict separation, demonstrating zero misclassifications among the three distinct operating systems. It is therefore evident that the MoE model is completely successful in mitigating negative transfer before the input data passes through the isolated 1D-CNN experts.

C. Overall Classification Performance

The empirical results definitively demonstrate the predictive capability and robustness of the proposed 1D-MoE architecture. Evaluated on a diverse testing set of 5,379 samples encompassing 31 distinct classes across three different operating systems, the model achieved an outstanding overall accuracy

TABLE VII
CLASSIFICATION REPORT FOR THE DYNAMIC GATING NETWORK. THE RESULTS DEMONSTRATE PERFECT PRECISION, RECALL, AND F1-SCORES (1.0000) ACROSS ALL 5,379 VALIDATION SAMPLES, CONFIRMING THE NETWORK'S FLAWLESS ABILITY TO IDENTIFY AND ROUTE ANDROID, LINUX, AND WINDOWS BINARIES TO THEIR RESPECTIVE DOMAIN-SPECIFIC EXPERTS.

Class / Metric	Precision	Recall	F1-score	Support
Android	1.0000	1.0000	1.0000	1165
Linux	1.0000	1.0000	1.0000	2007
Windows	1.0000	1.0000	1.0000	2207
accuracy			1.0000	5379
macro avg	1.0000	1.0000	1.0000	5379
weighted avg	1.0000	1.0000	1.0000	5379

TABLE VIII
DETAILED MULTI-CLASS CLASSIFICATION REPORT FOR THE SPECIALIZED 1D-CNN EXPERTS. THE ARCHITECTURE ACHIEVES A WEIGHTED AVERAGE F1-SCORE OF 0.9646, DEMONSTRATING ROBUST PRECISION AND RECALL ACROSS HIGHLY HETEROGENEOUS WINDOWS, ANDROID, AND LINUX DOMAINS.

Label	Class	Precision	Recall	F1-score
0	Android_Adware_final	0.9615	0.9843	0.9728
1	Android_Banking_final	1.0000	0.9427	0.9705
2	Android_Benign_final	0.9721	1.0000	0.9858
3	Android_Riskware_final	0.9765	0.9765	0.9765
4	Android_Sms_final	0.9929	1.0000	0.9964
5	Linux_CoinMiner	0.8421	0.8205	0.8312
6	Linux_DDoSAgent	0.8750	0.7368	0.8000
7	Linux_Gafgyt	0.7006	0.6923	0.6964
8	Linux_Hajime	0.9531	0.9531	0.9531
9	Linux_Kaiji	1.0000	0.9722	0.9859
10	Linux_Mirai	0.9481	0.9606	0.9543
11	Linux_MooBot	0.5556	0.3333	0.4167
12	Linux_Prometei	1.0000	1.0000	1.0000
13	Linux_SSHdKit	1.0000	1.0000	1.0000
14	Linux_Tsunami	0.9259	0.7812	0.8475
15	Windows_agent	0.9556	0.9556	0.9556
16	Windows_alureon	0.9915	0.9750	0.9832
17	Windows_bho	0.9907	1.0000	0.9953
18	Windows_ceeinject	1.0000	0.9789	0.9894
19	Windows_cycbot	1.0000	1.0000	1.0000
20	Windows_delfinject	0.9815	0.9815	0.9815
21	Windows_fakerean	0.9669	0.9832	0.9750
22	Windows_hotbar	1.0000	1.0000	1.0000
23	Windows_obfuscator	0.9389	0.9685	0.9535
24	Windows_onlinegames	1.0000	0.9930	0.9965
25	Windows_rbot	0.9554	0.9817	0.9683
26	Windows_startpage	0.9903	0.9623	0.9761
27	Windows_vundo	1.0000	0.9895	0.9947
28	Windows_winwebsec	1.0000	1.0000	1.0000
29	Windows_zbot	1.0000	1.0000	1.0000
30	Windows_zeroaccess	1.0000	1.0000	1.0000
	Accuracy			0.9650
	Macro Avg	0.9508	0.9330	0.9405
	Weighted Avg	0.9646	0.9650	0.9646

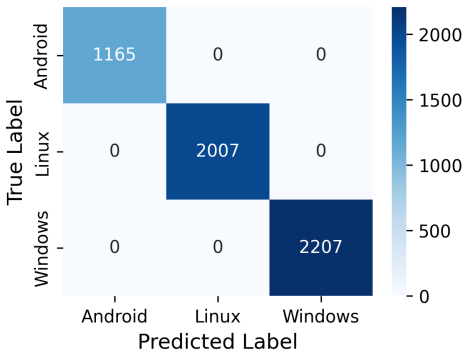


Fig. 5. Confusion matrix evaluating the gating network's routing performance. The strictly diagonal activation with zero off-diagonal misclassifications empirically validates that the network perfectly isolates heterogeneous operating system data, completely mitigating the risk of cross-domain negative transfer.

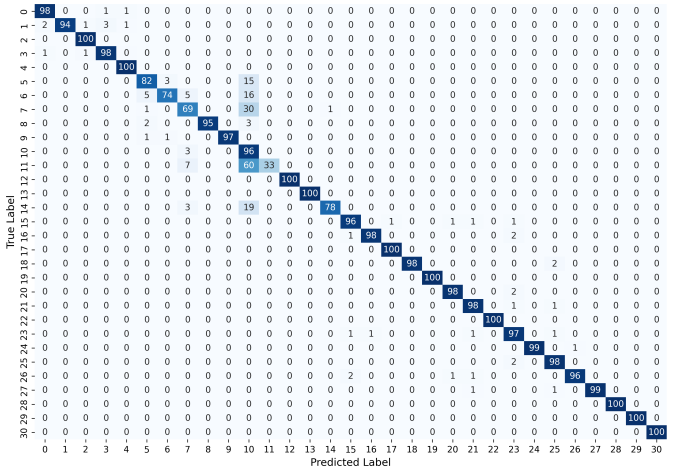


Fig. 6. Confusion matrix evaluating the performance of the 1D-CNN experts across all 31 malware and benign classes. The numerical axes correspond directly to the class labels detailed in Table VIII. The heavy concentration of activations along the primary diagonal demonstrates high classification fidelity with minimal inter-class confusion, validating the effectiveness of the 1D sequence extraction.

of 0.9650 and a weighted average F1-score of 0.9646 (see Table VIII).

The architecture proved exceptionally capable of learning malware structures without suffering from cross-domain negative transfer. As illustrated in the confusion matrix (Fig. 6), predictions remained tightly clustered along the true-positive diagonal. Several prevalent malware families, including *Windows_winwebsec*, *Windows_hotbar*, and *Linux_Prometei*, achieved perfect F1-scores of 1.0000. Even structurally complex classes like *Android_Sms_final* achieved near-perfect identification (F1 = 0.9964). While minor performance fluctuations were observed in heavily under-represented classes (e.g., *Linux_MooBot*), the aggregate metrics confirm that preserving the continuous 1D native sequence of executables, combined with dynamic MoE routing, results in a highly accurate and reliable cross-platform static analyzer.

D. Comparison with 2D

To empirically validate the structural advantages of preserving native byte sequences, the proposed 1D-MoE framework was benchmarked against a traditional 2D-CNN baseline. The 2D model utilized an identical parameter count, identical hyperparameter configurations, and the same testing split, but processed the executables as reshaped 2D square matrices padded with zeros.

As detailed in Table IX, the 2D baseline achieved a weighted average F1-score of 0.9082 and an overall accuracy of 0.9104. This represents a distinct degradation compared to the 1D model's weighted average F1-score of 0.9646. The performance disparity is most severe when analyzing the macro average F1-score, which plummeted from 0.9405 in the 1D architecture to 0.8322 in the 2D baseline.

The 2D model struggled significantly with complex or under-represented classes. For instance, the F1-score for *Windows_obfuscator* dropped from 0.9535 (1D) to 0.5979 (2D). More drastically, *Linux_Tsunami* fell from 0.8475 to just 0.1667, and the 2D model completely failed to identify any instances of *Linux_MooBot* (F1 = 0.0000). These results confirm our hypothesis: reshaping a continuous 1D byte stream into a 2D matrix introduces artificial spatial locality, forcing convolutional filters to learn fabricated relationships between adjacent rows. By avoiding these artificial spatial distortions, the segmented 1D approach proves vastly superior for cross-platform malware classification.

E. Ablation Study: Byte Segmentation

As detailed in the Methodology, the architecture discards the middle sections of the executables, processing only a concatenated sequence of the first and last 2^{16} bytes. An ablation study was performed to empirically validate this threshold.

To determine the optimal segment length, we evaluated the model on a subset of the dataset using various sequence lengths: 2^{14}, 2^{16}, 2^{18}, and 2^{20} bytes. As illustrated in Fig. 7, the 2^{16} byte length yielded the peak Macro and Weighted F1 scores. Performance degraded at larger sequence sizes, suggesting that extending the window forces the model to process an overwhelming amount of benign, structural noise.

Furthermore, to identify the most feature-dense regions of the binaries, we tested different positional segments, including the first, middle, and last 2^{16} byte chunks. The results demonstrated that the first and last segments contained the highest concentration of predictive malicious signatures. Consequently, our final architecture extracts and concatenates the first and last 2^{16} bytes to create a continuous 1D vector of length 131,072 (2×2^{16} bytes), as seen in Fig. 3. This segmentation process eliminates the artificial spatial distortions related to the use of 2D image representation while still preserving the native structure and high signal area that contributes the most to the classification.

F. Explainability and Interpretability Analysis

To move beyond aggregate quantitative metrics and interpret the model's decision-making process, 1D Grad-CAM was

TABLE IX
DETAILED MULTI-CLASS CLASSIFICATION REPORT FOR THE 2D-CNN ARCHITECTURE. EVALUATED ON THE IDENTICAL 5,379 SAMPLE TESTING SET, THE 2D MODEL ACHIEVES A LOWER WEIGHTED AVERAGE F1-SCORE OF 0.9082, EXHIBITING PERFORMANCE DEGRADATION IN MINORITY AND OBFUSCATED CLASSES DUE TO ARTIFICIAL SPATIAL DISTORTIONS.

Label	Class	Precision	Recall	F1-score
0	Android_Adware_final	0.8960	0.8819	0.8889
1	Android_Banking_final	0.9762	0.8542	0.9111
2	Android_Benign_final	0.8515	0.9885	0.9149
3	Android_Riskware_final	0.9355	0.9098	0.9225
4	Android_Sms_final	0.9695	0.9904	0.9798
5	Linux_CoinMiner	0.7778	0.5385	0.6364
6	Linux_DDoSAgent	0.6667	0.4211	0.5161
7	Linux_Gafgyt	0.5990	0.6982	0.6448
8	Linux_Hajime	0.8611	0.9688	0.9118
9	Linux_Kaiji	0.9710	0.9306	0.9504
10	Linux_Mirai	0.9225	0.9391	0.9307
11	Linux_MooBot	0.0000	0.0000	0.0000
12	Linux_Prometei	1.0000	1.0000	1.0000
13	Linux_SSHdKit	1.0000	0.9706	0.9851
14	Linux_Tsunami	0.7500	0.0938	0.1667
15	Windows_agent	0.8353	0.7889	0.8114
16	Windows_alureon	0.9344	0.9500	0.9421
17	Windows_bho	1.0000	0.9811	0.9905
18	Windows_ceeinject	0.9647	0.8632	0.9111
19	Windows_cycbot	1.0000	0.9900	0.9950
20	Windows_delfinject	0.8265	0.7500	0.7864
21	Windows_fakerean	0.9369	0.8739	0.9043
22	Windows_hotbar	1.0000	0.9521	0.9754
23	Windows_obfuscator	0.5305	0.6850	0.5979
24	Windows_onlinegames	0.9301	0.9366	0.9333
25	Windows_rbot	0.7965	0.8257	0.8108
26	Windows_startpage	0.8846	0.8679	0.8762
27	Windows_vundo	0.9179	0.9421	0.9299
28	Windows_winwebsec	0.9974	0.9947	0.9960
29	Windows_zbot	0.9934	0.9805	0.9869
30	Windows_zeroaccess	0.9913	0.9913	0.9913
	Accuracy			0.9104
	Macro Avg	0.8618	0.8245	0.8322
	Weighted Avg	0.9114	0.9104	0.9082

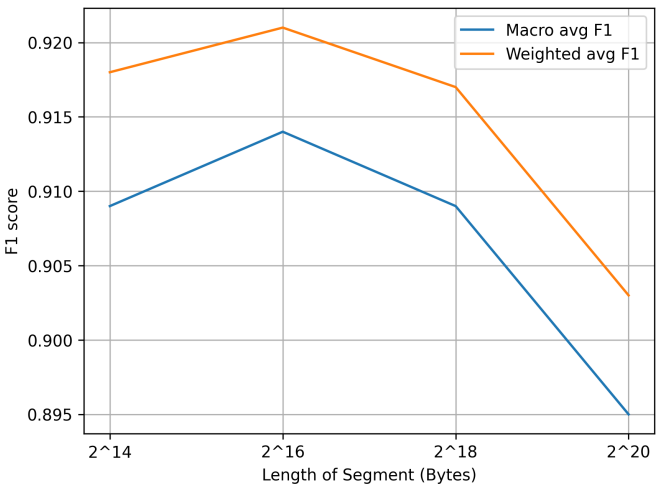


Fig. 7. Performance comparison of different segmentation lengths evaluated on a subset of the dataset. The 2^{16} byte threshold demonstrates the optimal balance, achieving the highest Macro and Weighted average F1 scores before performance degrades due to structural noise.

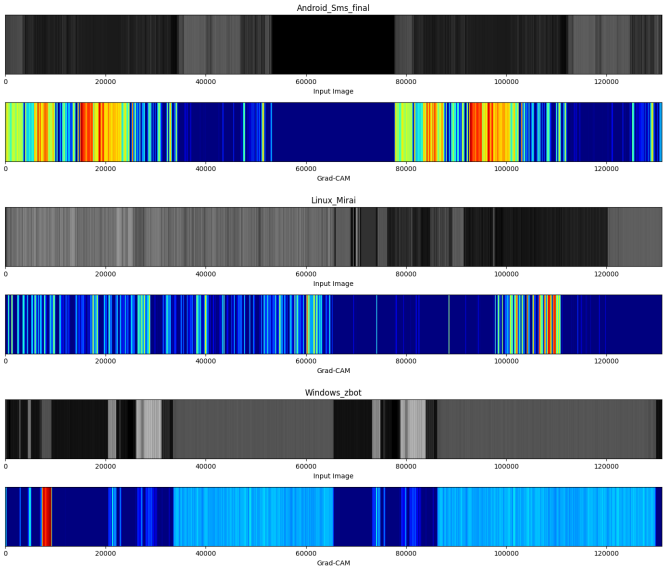


Fig. 8. Input image and corresponding GradCAMs of three samples, of *Android_Sms_final*, *Linux_Mirai*, and *Windows_zbot* classes, showing highly localized cluster of pixels with high importance in the final classification.

applied directly to the extracted 1D segments. This technique generates activation maps that are projected back onto the 2×2^{17} byte vector, visually isolating the specific sequences within the segments that heavily influenced the model's classifications.

Analysis of the resulting heatmaps consistently revealed high-magnitude activation spikes concentrated strictly within distinct, localized regions of the 1D sequence. Representative examples sampled across diverse domains—specifically *Android_Sms_final*, *Linux_Mirai*, and *Windows_zbot* (illustrated in Fig. 8)—demonstrate dense, highly localized clusters of predictive importance. This gradient-based evidence confirms that the 1D-CNN feature extractors are successfully identifying meaningful structural artifacts (such as payloads, headers, or import tables) within the extracted sequences, rather than relying on arbitrary byte distributions or structural noise.

G. Adversarial Resilience and Robustness

Building upon the interpretability insights provided by the 1D Grad-CAM visualizations, an ablation study was conducted to evaluate the architecture's resilience against targeted adversarial attacks. An obfuscation attack was simulated by masking the most highly predictive pixels—identified via localized Grad-CAM activation heatmaps—and subsequently re-evaluating the classification performance. The input sequences were modified by systematically zeroing out the top 5%, 10%, and 15% of the most contributing pixels.

As detailed in Table X, the model exhibits acute sensitivity to the perturbation of these localized, high-signal regions. From an optimal baseline weighted average F1-score of 0.9646, masking merely 5% of the critical bytes precipitates a drastic performance degradation to 0.6483 (a steep initial drop of 0.3163). Subsequent masking thresholds reveal a diminishing rate of degradation, with the F1-score falling to 0.5653

TABLE X
EVALUATION OF MODEL RESILIENCE AGAINST TARGETED ADVERSARIAL BYTE MASKING. THE RESULTS DEMONSTRATE A SEVERE INITIAL DEGRADATION IN THE WEIGHTED AVERAGE F1-SCORE WHEN MASKING JUST 5% OF THE HIGHEST-CONTRIBUTING BYTES, HIGHLIGHTING THE ARCHITECTURE'S HEAVY RELIANCE ON A HIGHLY LOCALIZED SET OF STRUCTURAL SIGNATURES.

Masked Bytes (%)	Weighted F1	Performance Drop
0	0.9646	Baseline
5	0.6483	-0.3163
10	0.5653	-0.0830
15	0.5196	-0.0457

at a 10% mask and 0.5196 at a 15% mask. These findings empirically validate that the 1D-CNN experts rely heavily on a highly concentrated set of structural features. While this hyper-focus ensures outstanding precision on pristine, unmodified binaries, it simultaneously exposes a critical vulnerability to targeted adversarial perturbations.

V. DISCUSSION AND LIMITATIONS

The evaluation metrics confirm that preserving the 1D sequence of raw bytecode provides a meaningful advantage over traditional 2D image reshaping. By avoiding the artificial grid imposed by 2D matrices, the 1D-CNN experts were able to capture the binaries' native structural relationships. Coupled with the Mixture of Experts (MoE) gating network, the architecture proved that cross-platform classification is viable without suffering from the negative transfer. The gating network functioned like a semantic firewall, ensuring the Android, Linux, and Windows experts processed only data relevant to their respective architectures.

However, the proposed methodology relies on data pruning strategy that introduces specific theoretical vulnerabilities. To solve the memory bloat associated with deep learning on raw executables, the preprocessing pipeline rigidly extracts only the first and last 2^{16} bytes, discarding the intermediate data.

Although the ablation study supported this choice for the current dataset, it relies on the assumption that malicious payloads or critical structural anomalies consistently appear near the start or end of the file. Advanced threat actors aware of this segmentation logic could theoretically bypass detection by injecting malicious payloads deep into the center of a heavily padded, benign executable. Because the current architecture is entirely blind to the middle segments of any binary exceeding 131,072 bytes, such a targeted evasion technique would succeed.

Additionally, as a purely static analysis tool, the framework evaluates the dormant structure of the files. It is structurally incapable of identifying behavior that only manifests at runtime, such as fileless malware, dynamic memory injection, or payloads fetched from remote command-and-control servers post-execution.

VI. CONCLUSION AND FUTURE DIRECTIONS

This research introduced a novel, high-performance architecture for cross-platform malware classification. By integrating a dynamic Mixture of Experts routing network the system effectively categorized Android DEX, Linux ELF, and Windows PE files without the accuracy degradation typically caused by negative transfer. Furthermore, by aggressively pruning benign structural noise and isolating the most feature-dense file extremities, the data pipeline radically reduced the computational overhead standardly associated with deep learning malware analysis.

Moving forward, future research should address the blind spots introduced by the static extraction threshold. One potential direction involves replacing the rigid 2^{16} byte cutoffs with an attention-based pre-scanner that dynamically identifies high-entropy regions across the entire binary, rather than relying on fixed positional boundaries. Additionally, expanding the MoE gating network to route and classify macOS Mach-O and other popular operating systems would push the architecture toward a truly universal, OS-agnostic static analyzer. Finally, combining this 1D static sequence extractor with a dynamic behavior-monitoring sandbox would bridge the gap between structural profiling and runtime threat detection.

REFERENCES

[1] House, "Malware daily trend," 2026.
[2] N. Daoudi *et al.*, "Dexray: A simple, yet effective deep learning approach to android malware detection based on image representation of bytecode," 2021.
[3] L. Nataraj *et al.*, "Malware images: visualization and automatic classification," in *Proceedings of the 8th international symposium on visualization for cyber security*, 2011, pp. 1–7.
[4] L. Chen *et al.*, "Stamina: A scalable deep learning approach for malware classification," 2020.
[5] R. R. Selvaraju *et al.*, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 618–626.
[6] A. Kumar *et al.*, "An efficient deep learning approach for malware classification," 2024.
[7] A. Kumar and R. Kumar, "MalSFF: Multi-static feature fusion for multi-architecture malware detection," 2026.
[8] R. A. Jacobs *et al.*, "Adaptive mixtures of local experts," *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.
[9] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," 2017.
[10] "CICMalDroid2020 dataset," [Online].
[11] A. Asmitha *et al.*, "Deep learning for windows malware classification," 2024.
[12] T. Akiba *et al.*, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.
[13] abuse.ch, "MalwareBazaar Malware Database," [Online]. Available: https://bazaar.abuse.ch/